

Anomaly Detection in Financial Institutions Using Isolation Forest

Yulisa M. Lopez Jeronimo

Intuition and Fundamentals

Introduction

Anomaly detection involves identification of relative unusual data . It plays a critical role in identifying fraudulent activities, in the context of this project credit card transactions, where the goal is to detect deviating patterns that may indicate fraudulent behavior in credit cards. One machine learning algorithm used for anomaly detection is Isolation Forest. Unlike traditional methods that rely on labeled data, Isolation Forest is an unsupervised technique designed to detect outliers (anomalies) by isolating data points that significantly differ from the majority. The algorithm works by constructing multiple decision trees to separate instances in the data set, and since anomalies are rare and distinct, they are more likely to be isolated quickly. In the context of credit card transactions, the target is to identify fraudulent transactions that deviate from normal spending patterns, which might involve unusual transaction amounts, locations, or time. Isolation Forest efficiently identifies these outliers, making it an ideal tool for detecting potential fraud in real-time without requiring extensive labeled data, enabling quicker responses to suspicious activities and enhancing security in financial systems.

Intuition

Anomaly detection (also called “outlier detection” or “novelty detection”) is the process of identifying data points that significantly differ from the rest of the set of observations . The “anomalies” are instances that do not conform to the expected pattern or behavior of the majority of the data. Depending on the specific method used, some assumptions underlying anomaly detection are that the majority of data points represent typical or normal behavior, while anomalies represent rare or unusual events that deviate from the norm. For example, in a card fraud detection system, most of the transactions are legitimate, but fraudulent transactions are anomalies that deviate from the usual patterns of the customer or a group.

In many anomaly detection methods, an anomaly (abnormal) data point is distant or significantly different from the rest of the data points. This could be in terms of distance or density; some data points are far away from the majority in feature space or they are in areas with very low data density (i.e., sparse regions in the data space).

Anomalies are often linked to rare events that do not conform to the usual patterns. In card fraud detection, most financial transactions are similar to the rest, but a high-value withdrawal from an unusual location could be an anomaly suggesting fraudulent activity. In addition, they have a low probability of occurring according to a certain distribution.

The type of anomaly detection covered in this paper is “Isolation Forest”; a tree-based algorithm which works by recursively partitioning data and isolating points that are different from the rest. Isolation Forest is particularly useful when dealing with large, high-dimensional data sets because it does not rely on distance or density-based approaches, which are often computationally expensive for large data sets.

The core idea of Isolation Forest is based on the assumption that anomalies (outliers) are few and different compared to the majority of the data (normal points), this makes it easier to isolate them with fewer splits. In contrast, normal data points are often more similar to other points and require more splits to isolate them. This difference in how many splits are needed to isolate the data points is what the algorithm uses to distinguish anomalies from normal points. This process is similar to building a decision tree, where data points are split into two subsets based on a random feature and a random split value. Since anomalous points are usually far from the bulk of the data, meaning that they tend to fall into smaller regions of the feature space quickly. Hence, fewer splits are required to isolate these outliers.

The isolation forest algorithm works by creating a series of random binary trees (called Isolation Trees) in which data points are recursively split at random feature values. Each tree is created by randomly selecting features and randomly choosing split points along those features. In which a short path length (i.e., fewer splits) in the tree suggests that the point is anomalous, while a longer path length (more splits) suggests that the point is normal. The path length is the number of splits required to isolate a point from the rest of the data in the tree.

To measure the performance of the model, an anomaly score is computed by averaging the path lengths across all trees in the forest. A short average path length across all trees indicates that the point is anomalous, while a longer average path length indicates that the point is more likely to be normal. This comes from the idea that points with shorter average path lengths are more easily isolated, and therefore have higher anomaly scores. The score ranges from 0 to 1, where values closer to 1 indicate anomalies and values closer to 0 indicate normal points.

Some of the advantages of Isolation Forest are scalability, good performance for high-dimensional data and parameter tuning is not as relevant. Isolation Forest is efficient for large data sets because it uses a small number of trees (compared to other methods that might require intensive calculations like k-nearest neighbors or clustering). As well, the algorithm doesn’t rely on calculating distances or densities, which can become computationally expensive in high-dimensional spaces. Lastly, the algorithm is relatively simple and doesn’t require a

lot of parameter tuning. Nonetheless, the number of trees and the depth of the trees are the primary hyperparameters, and their values don't need to be finely tuned.

The Isolation Forest algorithm is an efficient and effective anomaly detection technique, especially suited for large and high-dimensional data. Its intuition is based on the idea that anomalies are easy to isolate because they differ significantly from the normal data, which makes them stand out when partitioning the data. Calculating the path length required to isolate data points that allows for quick identification of anomalies within the data.

Fundamentals

How anomaly detection through Isolation Forest works:

To isolate a data point through a binary tree structure called Isolation Tree (iTree) the data will be repeatedly split by randomly chosen features and random split values between the minimum and maximum values allowed for a feature. This will generate the path length in the tree from the root to a terminating (external) node.

For example, in a set of d -dimensional points, where $\mathbf{X} = \{x_1, \dots, x_n\}$, a subset is $\{X' \subset \mathbf{X}\}$.

An iTree, which is a data structure, has the following properties:

Each of the nodes T in the iTree is either an external node with no other nodes or an internal node with one "test" and exactly two other nodes, T_l and T_r .

A test at the node T is made out of a feature q and a split value p such that the test $q < p$ will determine the line that intersects two or more other lines in the same plane at different points on either T_l or T_r .

The algorithm periodically divides X' through random selection of a variable q and a split value p , until either:

1. The node has only one observation
2. All the data at the node have the same values.

Once the iTree is done growing, each of the observations in $\mathbf{X}$ is isolated at one of the outer nodes. The ones with the smallest path length, $h(x_i)$, in the tree for the point $x_i \in \mathbf{X}$, is defined as the number of edges the point x_i traverses from the beginning of the tree to the last node.

Anomaly detection with Isolation Forest proceeds as follows:

Use the training dataset to build a number of iTrees.

For each data point in the test set:

Pass it through all the iTrees, counting the path length for each tree.

Assign an anomaly score to the point.

Label the point as an anomaly if its score is greater than a predefined threshold, which depends on the domain.

Anomaly Score

The algorithm for computing the anomaly score of a data point is based on the observation that the structure of iTrees is similar to Binary Search Trees (BST): a termination at an external node of the iTree corresponds to an unsuccessful search in the BST. Therefore, the estimation of the average $h(x)$ for external node terminations is equivalent to that of unsuccessful searches in a BST.

Specifically, the average path length $c(m)$ for a sample size m is given by:

$$c(m) = \begin{cases} 2H(m-1) - \frac{2(m-1)}{n} & \text{for } m > 2 \\ 1 & \text{for } m = 2 \\ 0 & \text{otherwise} \end{cases}$$

Where:

n is the test set size

m is the sample set size

$H(i) = \ln(i) + \gamma$, with $\gamma = 0.5772156649$ being the Euler-Mascheroni constant.

In this context, $c(m)$ represents the average path length $h(x)$ for a given sample size m , and it is used to normalize $h(x)$ to estimate the anomaly score for a given observation x .

The anomaly score $s(x, m)$ is then computed as:

$$s(x, m) = 2^{-\frac{E(h(x))}{c(m)}}$$

Where: $E(h(x))$ is the average path length $h(x)$ from a collection of iTrees.

Interpreting the Anomaly Score:

If $s(x)$ is close to 1, the point x is very likely an anomaly.

If $s(x)$ is smaller than 0.5, the point x is likely normal.

If all points in the sample have scores around 0.5, it is likely that they are all normal.

This process allows Isolation Forest to efficiently detect anomalies based on the ease with which a point can be isolated in the feature space.

Application of Isolation Forest for Anomaly Detection of Credit Card Transactions

The dataset comes from Kaggle:

Description of the variables of the dataset:

trans_date, trans_time – Timestamp of the transaction

cc_num – Unique (anonymized) credit card number

merchant – Merchant where the transaction occurred

category – Type of transaction (e.g., travel, food, personal care)

amt – Transaction amount

Cardholder Details:

first, last – First and last name of the cardholder

gender – Gender of the cardholder

street, city, state, zip' – Address of the cardholder

lat, long – Geographical location of the cardholder

city_pop – Population of the cardholder's city

job – Profession of the cardholder

dob – Date of birth of the cardholder

trans_num – Unique transaction identifier

unix_time – Timestamp of transaction in Unix format

Merchant Details:

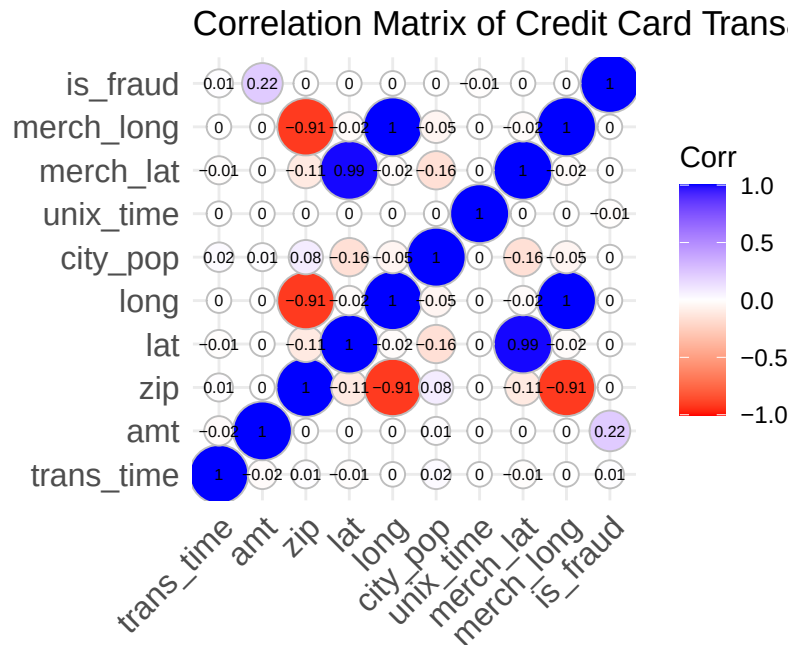
merch_lat, merch_long – Merchants location (latitude & longitude)

Fraud Indicator:

is_fraud – Target variable (1 = Fraud, 0 = Legitimate)

Visualizing the data

The correlation matrix below allows us to see the relationships between pairs of variables in the dataset. When two variables have a high correlation (close to 1 or -1) it indicates that they move together and we can remove (or combine them) to avoid multicollinearity. But in the case of creating an Isolation Forest it allows us to see which features can be removed without losing significant information to allow for a simpler model. Also, by removing them we can reduce dimensionality while improving our model's performance.

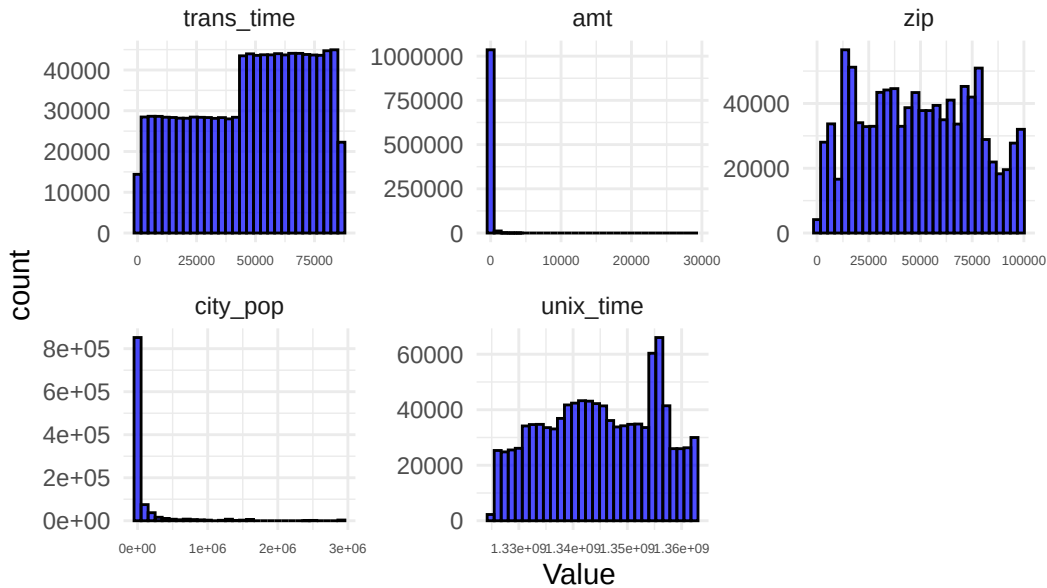


Eliminate some of the unnecessary variables that have high correlation such as long, merch_long, merch_lat, lat and the variables that are tend to be unique to individuals such as last, street, merchant, city, and trans_time (since we also have the unix time which also account for the date of the transaction not just the time).

Distribution of the Variables

The following graphs allows us to picture the distribution of the variables that we will use for out isolation forest and be able to see the major patterns that they have.

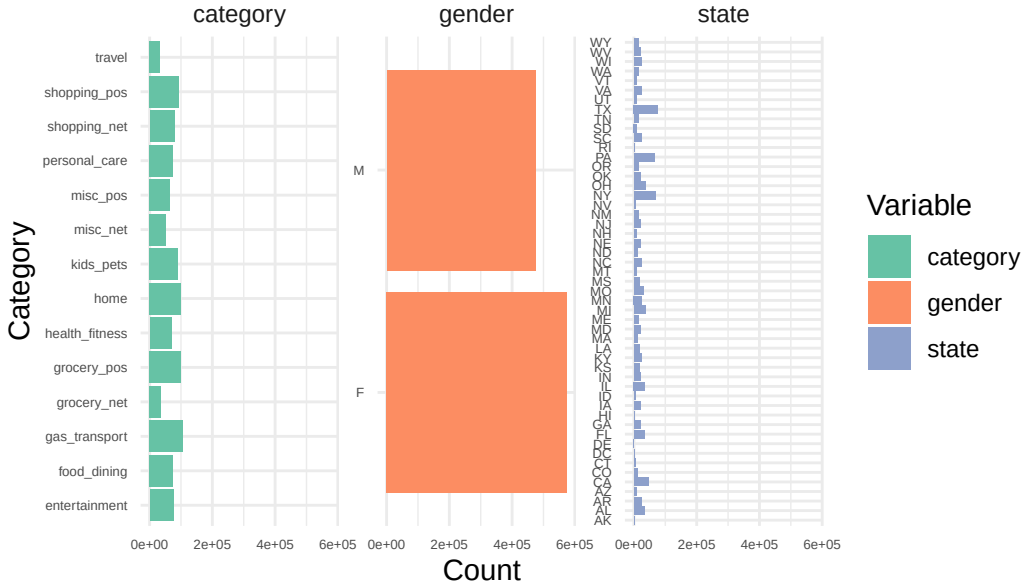
Distribution of Numeric Variables



Based on the picture above showing the distribution of the numeric variables variables that will be used. The distribution for the transaction time is based on seconds (in 24 hours there are 86400 seconds) in which appears to be roughly uniform with no strong peak times. ATM is the amount of the transaction in dollars, heavily skewed to the right, indicating that the majority of the transactions are small amounts and the long tail could indicate typical consumer behavior; most purchases with a credit card are low to moderate with few large ones. The distribution across zip codes is fairly uniform, the peaks at specific codes could reflect areas with higher transaction activity or more people with credit cards. The city- pop, describes the city population, which is skewed to the right, indicating that most transactions occur in cities with small populations. The unix time of transaction appears to have grouped peaks indicating that some periods have higher transaction frequencies such as burst of activity or time specific trends (holidays, weekends, etc.)

Distribution of all the categorical variables

Distribution of Categorical Variables



In the set of categorical variables, the (transaction) category distribution looks relatively even but `grocery_net`, `shopping_pos`, `gas_transport` and `shopping_net` seem more popular, indicating that the majority of the transactions are related to everyday purchases. The distribution for gender is almost perfectly balanced between male and female categories. For the state distribution, CA, TX, NY and FL have much higher count of transactions, likely reflecting the population size or sampling concentration in the dataset.

The data frame below shows the proportion (`prop_train`) and the number (`count_train`) of the transactions that are legitimate and the ones that are fraudulent in the whole dataset. At first glance we can observe that there is less than 1% of the transactions that are labeled as fraud compared to the rest of the data.

Table 1: Distribution of Fraud and Legitimate Transactions

	Transaction Type	Percentage (%)	Count
0	Legitimate	99.43	1042569
1	Fraud	0.57	6006

Isolation Forest with the package `isotree`

First, some of the hyperparameters that could be tuned are `ntrees` that stands for the number of trees, `max_depth` is the maximum depth of the binary tree to grow, the standard measure

is used in this example. Some other argument used are `dim` which is the number of columns to combine to produce a split, and `scoring_metric` which is the selected metric to use for determining outlier scores.

The isolation forest will produce the anomaly score ranging from 0 to 1 as explained on the intuition and fundamentals sections.

1 : it can be interpreted as an anomaly.

0.5 : it can be confirmed that there is no anomaly for the sampling dataset.

< 0.5 : It can be interpreted as a regular point.

To analyze the performance of the algorithm, we will use confusion matrix. The confusion matrix will be focusing on precision over the recall since we will try to minimize the normal customer wrongly categorized as customer with fraudulent transaction. Otherwise the bank may loss the valuable customer for their business target.

Before continuing into confusion matrix, we will divide the anomaly score with the a specific threshold. If the anomaly score is more than 80% quantile for the rest of data, we will classify the point as an anomaly. This threshold can be tolerated with respective subject business matter.

	0%	80%
	0.2905683	0.3334917

Now we add a new column labeled `fraud_detection` that is 1 = fraud if the `fraud_score` is greater or equal to the 80% quantile or 0 = legitimate if its less less than the 80% quantile.

Now we perform the confusion matrix:

The results below are:

True Negatives (TN): 837521 → Correctly identified legitimate transactions.

False Positives (FP): 1,338 → Misclassified legitimate transactions as fraud.

False Negatives (FN): 205,048 → Huge number of missed fraud cases.

True Positives (TP): 4,668 → Correctly detected fraud cases.

However, the model has an accuracy of 80.3% which seems high, but the value is inflated due to the majority of the transactions being legitimate. In that case, accuracy alone is not a good measure in such an imbalanced dataset. In addition, precision (Positive Predictive Value for Fraud) is 77.7% , when the model predicts fraud, it is correct 77.7% of the time, and balanced Accuracy is 51.03%, just slightly better than random guessing (50%)

Confusion Matrix and Statistics

```

      Reference
Prediction    0      1
0 920599 121970
1   1875   4131

      Accuracy : 0.8819
      95% CI   : (0.8813, 0.8825)
No Information Rate : 0.8797
P-Value [Acc > NIR] : 5.645e-12

      Kappa   : 0.0522

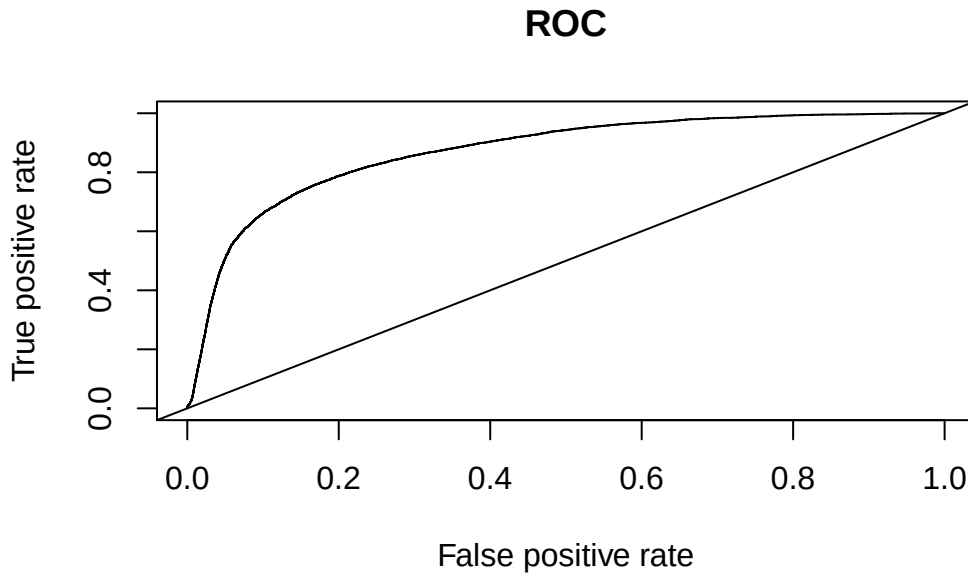
McNemar's Test P-Value : < 2.2e-16

      Sensitivity : 0.032759
      Specificity : 0.997967
Pos Pred Value   : 0.687812
Neg Pred Value   : 0.883010
Prevalence       : 0.120259
Detection Rate   : 0.003940
Detection Prevalence : 0.005728
Balanced Accuracy : 0.515363

'Positive' Class : 1
```

Overall this could mean that, the model is heavily biased toward legitimate transactions. It rarely catches fraud (low recall), meaning many fraudulent transactions go undetected. The high precision (77.7%) means that when the model does predict fraud, it is often right, but this comes at the cost of missing nearly all fraud cases.

In addition, we can perform an ROC curve (receiver operating characteristic curve) to show the performance of this classification model at different classification thresholds to evaluate the performance of the model.

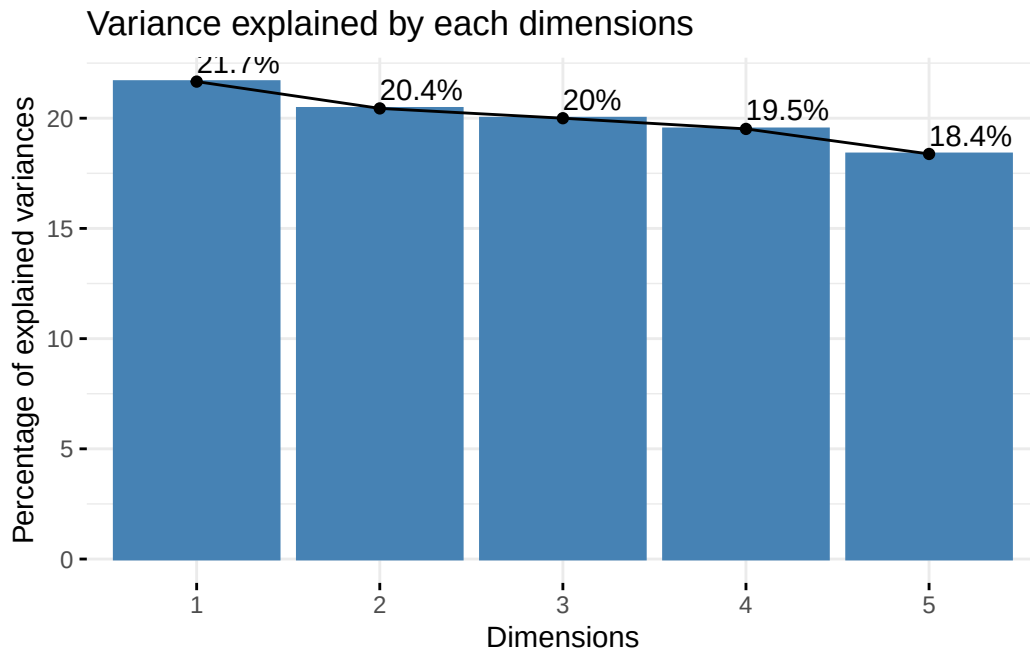


The ROC curve is well above the diagonal line which means that the model has predictive power. At the beginning it starts steeply, indicating good early separating of fraud and legitimate transactions. However, it flattens out, which means that further improvements come at the cost of more false positives. This ROC curve suggests an AUC (Area under the curve) likely between 0.80 to 0.85, which is decently strong.

To be sure, we calculated the AUC to evaluate the performance of our binary classification model (Isolation Forest) is 0.858, which is a good result since the more AUC is close to 1, it means that the prediction performance is better.

Visualizing the Isolation Forest Score

A plot will be needed to visualize the score produced by the Isolation Forest algorithm. The visualization will give us more insight into the polarization of the anomaly score. Since the visualization is better with fewer dimensions, we will compress the data into the least possible dimensions using PCA; then, we will take the dimensions that explain most of the variation in the data produced by the PCA and visualize it in a plot. I am choosing PCA because of its main application of dimensionality reduction to reduce the number of variables (dimensions) in the dataset while retaining as much of the variance (information) as possible.



The relative majority of the data seems to be retained by the first three dimensions which is 62.1%.

Also, we will create a plot for the spreading of the anomaly score. By using the first three dimensions, we create the Isolation Forest and visualize the anomaly score.

	d1	d2	d3
1	-1.8208001	-1.67721	-2.11088
2	-1.6279869	-1.67721	-2.11088
3	-1.4351737	-1.67721	-2.11088
4	-1.2423605	-1.67721	-2.11088
5	-1.0495473	-1.67721	-2.11088
6	-0.8567341	-1.67721	-2.11088

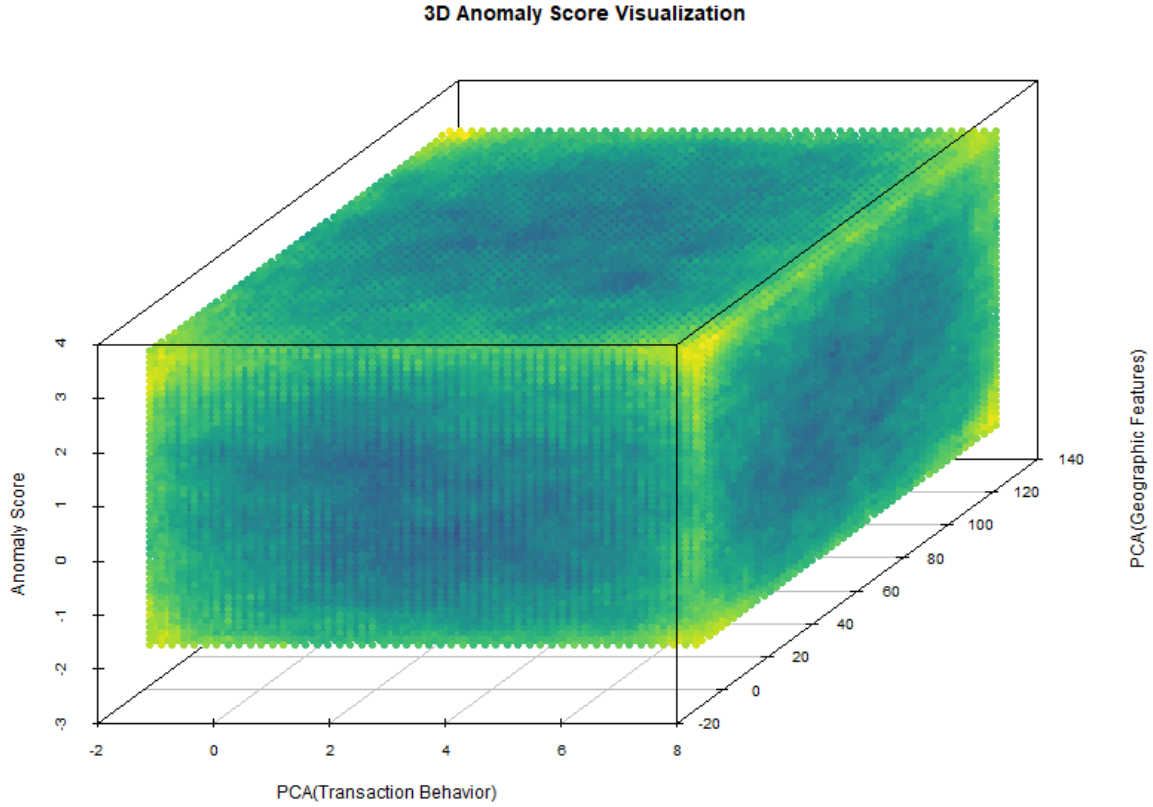
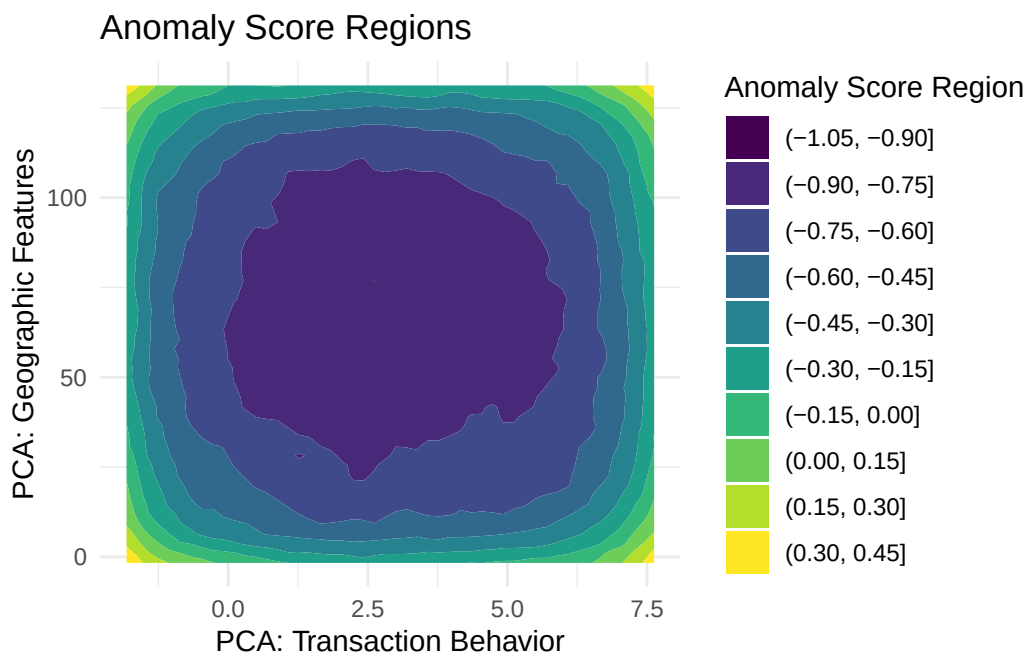


Figure 1: 3D Anomaly Score Visualization

The transaction behavior axis represents the first principal component, capturing transaction patterns such as amount, time and frequency. The geographic feature axis represents the second principal component, capturing the location based variance such as zip code and city population. The anomaly score is is the isolation forest model, in which higher values indicate greater likelihood of fraudulent transactions. The yellow regions are higher anomaly scores, indicating fraudulent activity while green/ blue regions are lower anomaly scores, representing normal transaction behavior. The concentrated yellow areas suggest that specific transaction behaviors or geographic patterns are more likely to be fraudulent.

In case that we were interested in observing the 2D version of the graph above to identify any potential patterns that we should be aware of when looking for any fradulent transactions, the picture below can help us. To obtain a clean 2D visualization with a large dataset I created 10 groups per dimension (anomaly score, geographic features and transaction behavior), in which the horizotal axis represents the transaction behavior and the vertical axis represents the geographic features, since the anomaly score will be represented by the legend label. From

this graph we learn that the outer regions, where the anomaly score is high, a potential fraud is more likely to occur. In the central dark area, the anomaly scores are the lowest, indicating that most transactions fall into a common pattern. Though, the smooth gradient from center to edges indicates that the deviation from what is considered normal behavior is continuous, not abrupt. This could suggest that some borderline behaviors exist between legitimate and anomalous, rather than binary categories. This graph confirms that the Isolation Forest is able to detect outliers based on sparse distribution in feature space, observations farther from the center are flagged as fraudulent more.



Conclusion

In this study, we explored the application of the Isolation Forest algorithm for anomaly detection in financial transactions, specifically for identifying fraudulent credit card activities. The Isolation Forest technique proves to be highly effective for detecting anomalies due to its ability to isolate outliers efficiently, even in high-dimensional data sets. Unlike traditional distance or density-based approaches, which can be computationally expensive, Isolation Forest leverages a tree-based structure to differentiate normal transactions from fraudulent ones with minimal computational overhead.

Our analysis demonstrated that fraudulent transactions exhibit distinct patterns, such as unusual transaction amounts, locations, or timestamps, making them easily detectable through the algorithm's partitioning mechanism. Additionally, preprocessing steps such as feature

selection, correlation analysis, and data visualization played a crucial role in refining the dataset and improving the model's performance.

Despite its advantages, Isolation Forest is not without limitations. The method relies on the assumption that anomalies are sparse and significantly different from normal data points. In cases where fraudulent transactions closely resemble legitimate ones, additional techniques such as ensemble learning or hybrid models may be necessary to enhance detection accuracy. Future work could also explore integrating Isolation Forest with supervised learning approaches to improve classification performance using labeled data.

Overall, this research highlights the potential of Isolation Forest as a scalable and efficient solution for real-time fraud detection in financial institutions. By leveraging this unsupervised learning approach, financial institutions can strengthen security measures, reduce financial losses, and enhance trust in digital payment systems.

Resources

Cortés, D. (n.d.). isotree: Isolation-based anomaly detection and predictive modeling. GitHub. Retrieved March 13, 2025, from <https://github.com/david-cortes/isotree>

Liu, F. T., Ting, K. M., & Zhou, Z. H. (2021). Isolation forest: A simple yet efficient approach to anomaly detection. *Pattern Recognition Letters*, 144, 37-44. <https://doi.org/10.1016/j.patrec.2021.06.015>

Zhou, C. (n.d.). Isolation forest & cats: Unsupervised anomaly detection (Part 4, R & Python). Medium. Retrieved March 13, 2025, from <https://medium.com/@christina.zhou.96/isolation-forest-cats-unsupervised-anomaly-detection-part-4-r-python-6d82cef50a66>

Zhou, C., & Cortes, D. (2018). Isolation Forest: Anomaly Detection for Unsupervised Learning. *Proceedings of the ACM on Knowledge Discovery and Data Mining*, 22(3), 44-57. <https://doi.org/10.1145/3338840.3355641>